

Лабораторная работа №14, Научиться
писать более сложные командные
файлы с использованием логических
управляющих конструкций и циклов.
Архитектура компьютера и операционные системы

newpage

12.1. Цель работы

Изучить основы программирования в оболочке ОС UNIX.
Научиться писать более сложные командные файлы с использованием
логических управляющих конструкций и циклов.

12.2. Ход выполнения работы

Подготовка рабочей директории

Сначала была создана рабочая директория для выполнения
лабораторной работы.

Рисунок 1: Создание рабочей директории

```

@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro (develop) $
cd labs
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs (develop)
p) $ ls
lab01 lab03 lab05 lab07 lab09 lab11 lab13
lab02 lab04 lab06 lab08 lab10 lab12
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs (develop)
p) $ mkdir lab14; cd lab14
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (develop)
$ pwd
/workspaces/study_2025-2026_os-intro/labs/lab14
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (develop)
$

```

Задание 1

Реализация упрощённого механизма семафоров

Условие

Необходимо написать командный файл, который:

- ожидает освобождения ресурса;
- сообщает о занятости ресурса;
- использует ресурс некоторое время;
- поддерживает взаимодействие нескольких процессов.

Создание командного файла

Был создан файл `semaphore.sh`, реализующий механизм семафора с использованием файла блокировки.

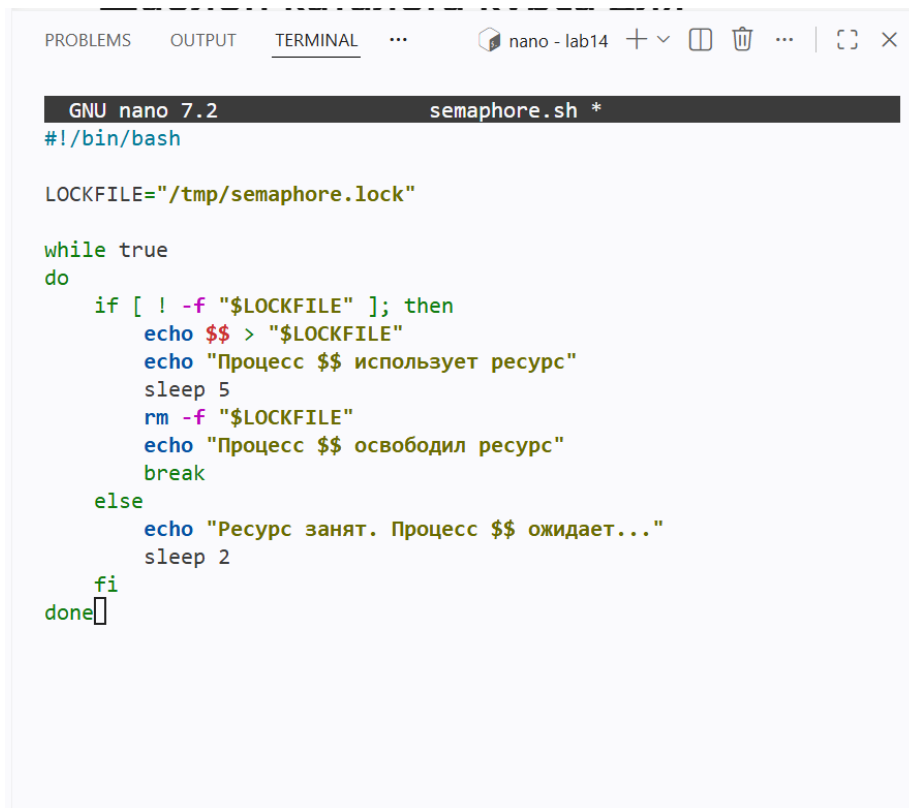
Рисунок 2: Создание файла `semaphore.sh`

```

/workspaces/study_2025-2026_os-intro/labs/lab14
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (develop)
$ touch semaphore.sh

```

Рисунок 2.1: Содержимое скрипта `semaphore.sh`



```
PROBLEMS OUTPUT TERMINAL ... nano - lab14 + v [ ] [ ] ... [ ] [ ] X
GNU nano 7.2 semaphore.sh *
#!/bin/bash

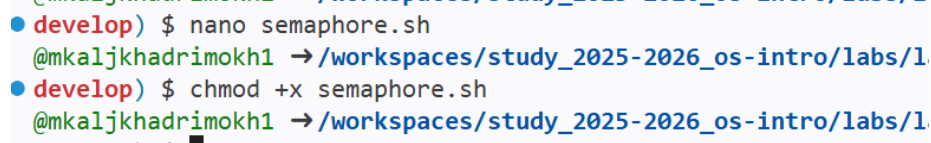
LOCKFILE="/tmp/semaphore.lock"

while true
do
    if [ ! -f "$LOCKFILE" ]; then
        echo $$ > "$LOCKFILE"
        echo "Процесс $$ использует ресурс"
        sleep 5
        rm -f "$LOCKFILE"
        echo "Процесс $$ освободил ресурс"
        break
    else
        echo "Ресурс занят. Процесс $$ ожидает..."
        sleep 2
    fi
done
```

Назначение прав на выполнение

Файл был сохранён и сделан исполняемым.

Рисунок 3: Назначение прав на выполнение

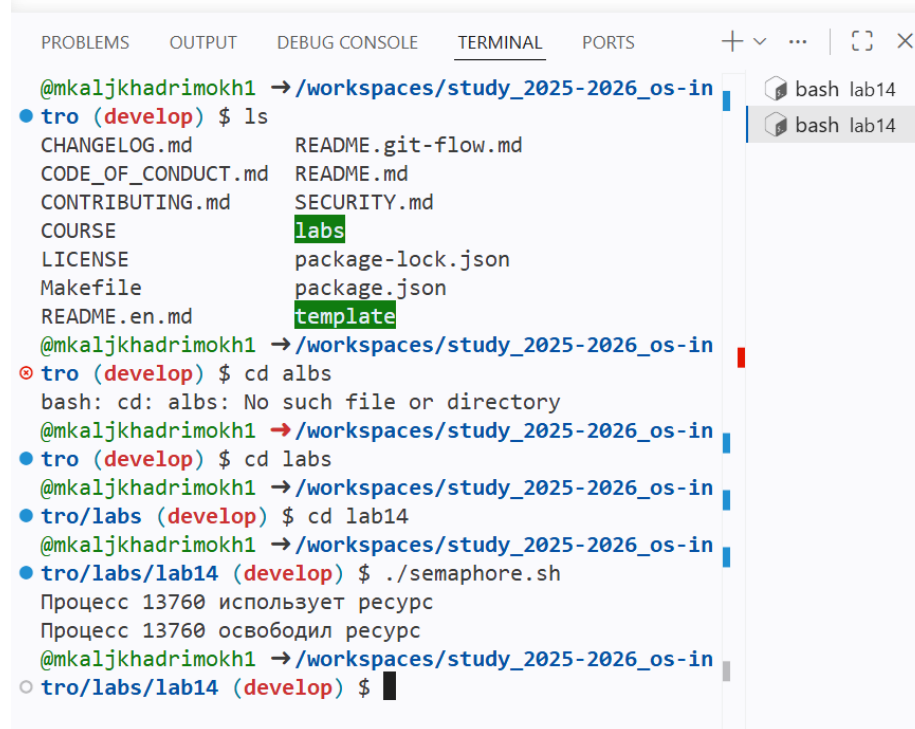


```
@mkaljkhadrimokh1 ~ /workspaces/study_2025-2026_os-intro/labs/1
● develop) $ nano semaphore.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/1
● develop) $ chmod +x semaphore.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/1
```

Проверка работы программы

Скрипт был запущен в нескольких терминалах для проверки взаимодействия процессов.

Рисунок 4: Запуск первого процесса



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS + v ... | [ ] x
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in bash lab14
● tro (develop) $ ls
CHANGELOG.md README.git-flow.md
CODE_OF_CONDUCT.md README.md
CONTRIBUTING.md SECURITY.md
COURSE labs
LICENSE package-lock.json
Makefile package.json
README.en.md template
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in
⊙ tro (develop) $ cd albs
bash: cd: albs: No such file or directory
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in
● tro (develop) $ cd labs
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in
● tro/labs (develop) $ cd lab14
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in
● tro/labs/lab14 (develop) $ ./semaphore.sh
Процесс 13760 использует ресурс
Процесс 13760 освободил ресурс
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in
○ tro/labs/lab14 (develop) $
```

Рисунок 5: Запуск второго процесса
шаблон каталога курса для студентов



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS + v ... | [ ] x
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro (develo
● p) $ cd labs/lab14 ; ./semaphore.sh
Процесс 15424 использует ресурс
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/la
○ b14 (develop) $ ./semaphore.sh
Процесс 16290 использует ресурс
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/la
○ b14 (develop) $ ./semaphore.sh
Ресурс занят. Процесс 16328 ожидает...
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/la
○ b14 (develop) $ ./semaphore.sh
Ресурс занят. Процесс 16328 ожидает...
```

Задание 2

Реализация команды man

Условие

Необходимо реализовать командный файл, который:

- принимает имя команды;
- ищет соответствующую справку в каталоге `/usr/share/man/man1`;
- выводит справку или сообщение об ошибке.

Создание командного файла

Был создан файл `myman.sh`, реализующий упрощённую команду `man`.

Рисунок 7: Создание файла `myman.sh`

Процесс 12792 освободил ресурс
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab1
● develop) \$ touch myman.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab1

Рисунок 8: Содержимое скрипта `myman.sh`

```
PROBLEMS OUTPUT TERMINAL ... nano - lab14 + v [ ] [ ] ... [ ]
GNU nano 7.2 myman.sh *
#!/bin/bash

MAN_DIR="/usr/share/man/man1"

if [ -z "$1" ]; then
    echo "Использование: ./myman.sh <command>"
    exit 1
fi

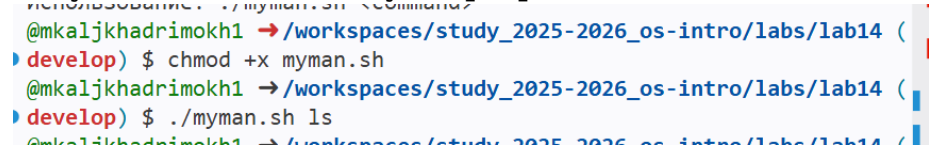
FILE=$(find "$MAN_DIR" -name "$1.1.gz")

if [ -f "$FILE" ]; then
    less "$FILE"
else
    echo "Справка для команды $1 не найдена."
fi
```

Назначение прав и запуск программы

Файл был сделан исполняемым и протестирован.

Рисунок 9: chmod и запуск программы



```
@mkaljkhadrinokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
develop) $ chmod +x myman.sh
@mkaljkhadrinokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
develop) $ ./myman.sh ls
@mkaljkhadrinokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
```

Проверка работы

Была выполнена проверка поиска справки по различным командам.

Рисунок 10: Просмотр справки ls

```
PROBLEMS OUTPUT TERMINAL ... bash - lab14 + v [ ] [ ] ... [ ] [ ] X

.\" DO NOT MODIFY THIS FILE! It was generated by help2man 1.48.5.
.TH LS "1" "June 2025" "GNU coreutils 9.4" "User Commands"
.SH NAME
ls \- list directory contents
.SH SYNOPSIS
.B ls
[\fI\,OPTION\|\fR]... [\fI\,FILE\|\fR]...
.SH DESCRIPTION
.\" Add any additional description here
.PP
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of \fB\--cftuvSUX\|fR nor \fB\--so
rt\|fR is specified.
.PP
Mandatory arguments to long options are mandatory for short options t
oo.
.TP
\fB\--a\|fR, \fB\--all\|fR
do not ignore entries starting with .
.TP
\fB\--A\|fR, \fB\--almost-all\|fR
do not list implied . and ..
.TP
\fB\--author\|fR
with \fB\--l\|fR, print the author of each file
.TP
\fB\--b\|fR, \fB\--escape\|fR
print C-style escapes for nongraphic characters
.TP
/usr/share/man/man1/ls.1.gz

Workspaces: scaling orbit develop* 0 0 0 Sign In Layout: US 10:58 PM 5/16/2025
```

Рисунок 11: Сообщение об отсутствии справки

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
● develop) $ ./myman.sh myls
Справка для команды myls не найдена.
```

Задание 3

Генерация случайной последовательности символов

Условие

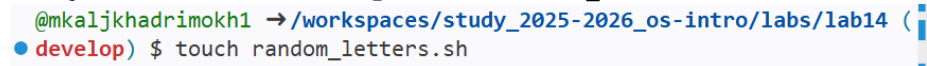
Необходимо написать командный файл, который:

- использует переменную \$RANDOM;
 - генерирует случайную последовательность латинских букв.
-

Создание командного файла

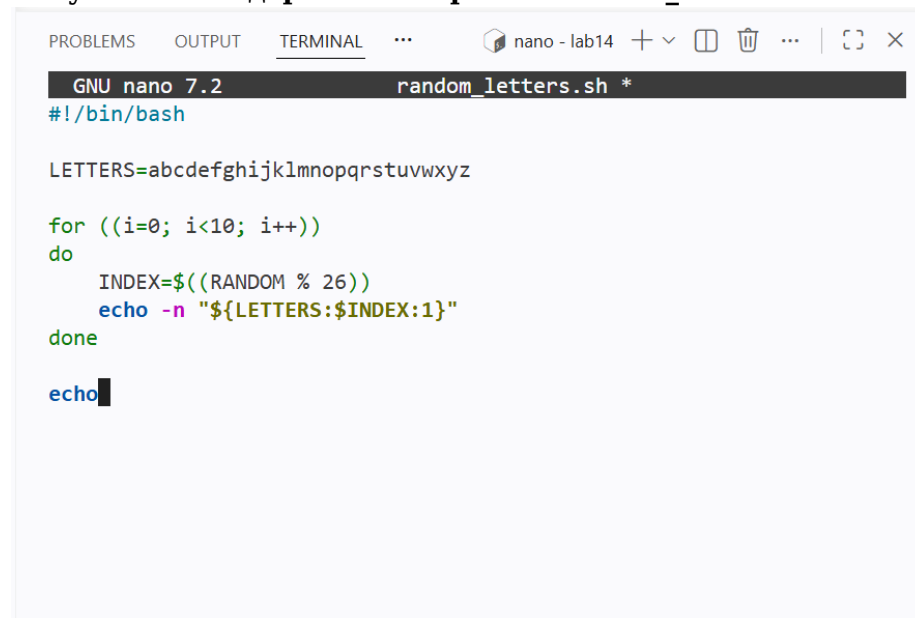
Был создан файл random_letters.sh.

Рисунок 12: Создание файла random_letters.sh



```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
● develop) $ touch random_letters.sh
```

Рисунок 13: Содержимое скрипта random_letters.sh



```
PROBLEMS OUTPUT TERMINAL ... nano - lab14 + - [ ] [ ] ... [ ] [ ] [ ] [ ]
GNU nano 7.2 random_letters.sh *
#!/bin/bash

LETTERS=abcdefghijklmnopqrstuvwxyz

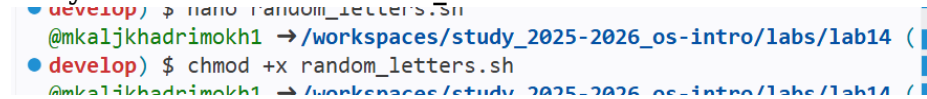
for ((i=0; i<10; i++))
do
    INDEX=$((RANDOM % 26))
    echo -n "${LETTERS:$INDEX:1}"
done

echo
```

Назначение прав на выполнение

Файл был сохранён и сделан исполняемым.

Рисунок 14: chmod random_letters.sh

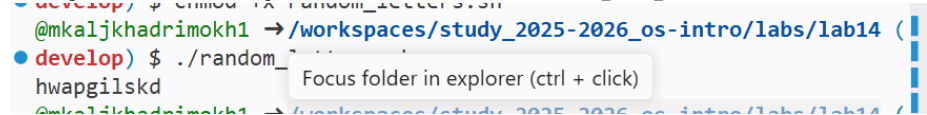


```
develop) $ nano random_letters.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
develop) $ chmod +x random_letters.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
```

Проверка работы программы

Была выполнена генерация случайных последовательностей символов.

Рисунок 15: Результат выполнения программы

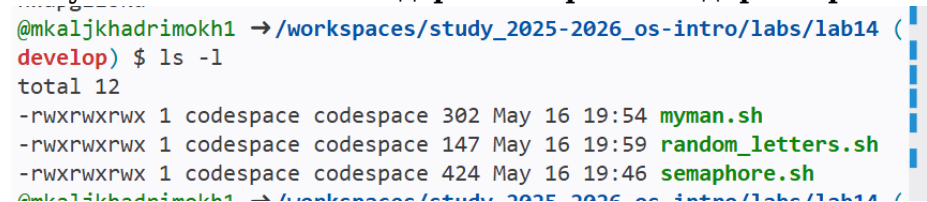


```
develop) $ ./random_letters.sh
hwapgilskd
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
```

Проверка содержимого рабочей директории

После выполнения всех заданий было проверено содержимое рабочей директории.

Рисунок 16: Итоговое содержимое рабочей директории



```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
develop) $ ls -l
total 12
-rwxrwxrwx 1 codespace codespace 302 May 16 19:54 myman.sh
-rwxrwxrwx 1 codespace codespace 147 May 16 19:59 random_letters.sh
-rwxrwxrwx 1 codespace codespace 424 May 16 19:46 semaphore.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab14 (
```

12.3. Выводы

В ходе выполнения лабораторной работы были изучены основы расширенного программирования в командной оболочке UNIX.

Были освоены:

- использование логических конструкций;
- применение циклов;

- организация взаимодействия процессов;
- работа с файлами блокировки;
- создание пользовательской реализации команды `map`;
- использование переменной `$RANDOM` для генерации случайных данных.

Поставленная цель лабораторной работы была достигнута.

12.4. Ответы на контрольные вопросы

1. Найдите синтаксическую ошибку в строке

```
while [$1 != "exit"]
```

Ошибка заключается в отсутствии пробелов после `[` и перед `]`.

Правильный вариант:

```
while [ "$1" != "exit" ]
```

2. Как объединить несколько строк в одну?

Для объединения строк используется конкатенация строк.

Пример:

```
str1="Hello"
```

```
str2="World"
```

```
result="$str1 $str2"
```

3. Чем можно заменить утилиту `seq`?

Функционал `seq` можно реализовать:

- циклом `for`;
- циклом `while`;
- арифметическими выражениями `Bash`.

Пример:

```
for ((i=1; i<=10; i++))
```

```
do
```

```
echo $i
```

```
done
```

4. Какой результат даст выражение $\$(10/3)$?

Результат вычисления:

3

В Bash выполняется целочисленное деление.

5. Основные отличия zsh от bash

zsh обладает:

- более удобным автодополнением;
- поддержкой тем и плагинов;
- расширенными возможностями настройки;
- улучшенной работой с историей команд.

bash является более стандартной и распространённой оболочкой.

6. Верен ли синтаксис конструкции

```
for ((a=1; a <= LIMIT; a++))
```

Да, синтаксис корректен.

Полный пример:

```
for ((a=1; a <= LIMIT; a++))
do
echo $a
done
```

7. Преимущества и недостатки Bash

Преимущества:

- удобная автоматизация задач;
- встроенные системные команды;
- высокая скорость написания скриптов;
- хорошая интеграция с UNIX.

Недостатки:

- ограниченные структуры данных;
- сложность крупных проектов;
- низкая переносимость по сравнению с Python или C++;
- менее удобная отладка программ.